

AI-Powered Movie Review Movie Analyzer

An end-to-end deep learning NLP project that classifies movie reviews as **positive** or **negative** using a fine-tuned DistilBERT transformer model which is deployed as a live interactive web app.

 Python 3.10+

 PyTorch 2.x

 HuggingFace Transformers

 Streamlit deployed

License MIT

Overview

This project demonstrates a complete machine learning pipeline, starting from raw data exploration ending in a deployed web application. It uses the [IMDB dataset](#) (50,000 movie reviews) and fine-tunes a **DistilBERT** model to classify sentiment with ~93% accuracy.

Live Demo

[Try it on Hugging Face Spaces](#)

Results

Metric	Score
Accuracy	93.2%
F1 Score	0.932
Precision	0.934
Recall	0.930

Evaluated on 25,000 held-out IMDB test reviews.

Project Structure

```
movie_classifier/  
├── data/  
│   └── raw/                # Original IMDB dataset  
└── notebooks/
```

```

├── 01_EDA.ipynb          # Exploratory data analysis
├── 02_training.ipynb     # Model training walkthrough
├── src/
│   ├── preprocess.py     # Tokenization & data loading
│   ├── train.py          # Training loop & config
│   ├── evaluate.py       # Metrics & confusion matrix
│   └── predict.py        # Inference on new reviews
├── app/
│   └── app.py            # Streamlit web application
├── models/
│   └── best_model.pt      # Saved fine-tuned model
├── assets/
│   └── demo_screenshot.png # App screenshot for README
├── requirements.txt
└── README.md

```

Tech Stack

Layer	Tool / Library
Language	Python 3.10+
Deep Learning	PyTorch 2.x
NLP Model	DistilBERT (HuggingFace)
Data	HuggingFace datasets
Web App	Streamlit
Deployment	Hugging Face Spaces
Experiment Tracking	(optional) Weights & Biases

How to Run Locally

1. Clone the repository

```

git clone https://github.com/johannesfelzmann/movie_classifier.git
cd movie_classifier

```

2. Create a virtual environment

```
python -m venv venv
source venv/bin/activate      # On Windows: venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Train the model

```
python src/train.py
```

Training takes ~20 minutes on a GPU, ~2 hours on CPU.

A pre-trained checkpoint is available in `/models/best_model.pt`.

5. Launch the web app

```
streamlit run app/app.py
```

Open <http://localhost:8501> in your browser.

Model Architecture

This project fine-tunes **DistilBERT** (`distilbert-base-uncased`), a lightweight version of BERT that retains 97% of BERT's language understanding while being 40% smaller and 60% faster.

```
Input Review Text
    ↓
DistilBERT Tokenizer (max 512 tokens)
    ↓
DistilBERT Encoder (6 transformer layers)
    ↓
[CLS] token representation
    ↓
Linear Classification Head (768 → 2)
    ↓
Softmax → Positive / Negative
```

Training config:

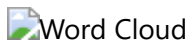
- Optimizer: AdamW

- Learning rate: 2e-5 with linear warm-up
 - Batch size: 16
 - Epochs: 3
 - Loss: CrossEntropyLoss
-

Exploratory Data Analysis

Key findings from [notebooks/01_EDA.ipynb](#):

- Dataset is **perfectly balanced** (25k positive, 25k negative)
- Average review length: **~230 words**
- Most common positive words: *great, excellent, wonderful, best, amazing*
- Most common negative words: *bad, worst, waste, boring, awful*



Data

The project uses the [IMDB Large Movie Review Dataset](#):

- **50,000** labeled movie reviews
- **25,000** for training, **25,000** for testing
- Binary labels: positive (1) / negative (0)

To download via HuggingFace:

```
from datasets import load_dataset
dataset = load_dataset("imdb")
```

Future Improvements

- ☐ Add **confidence score** display in the UI
 - ☐ Support **batch predictions** from CSV upload
 - ☐ Track experiments with **Weights & Biases**
 - ☐ Store prediction logs in **PostgreSQL** database
 - ☐ Fine-tune on domain-specific data (product reviews, tweets)
 - ☐ Add **explainability** with SHAP or attention visualization
-

License

This project is licensed under the **MIT License** — see the [LICENSE](#) file for details.

Author

Your Name

- GitHub: [@johannes_felzmann](#)
- LinkedIn: [linkedin.com/in/YOUR_PROFILE](#)

If you found this project useful, consider giving it a star!